

Herramienta de Diagnóstico de Sensor MPU6050 mediante Identificación de ID Interno en Plataforma ESP32

Carlos Daniel Cuellar Antury

Departamento de Ingeniería Electrónica y Telecomunicaciones, Universidad del Cauca, Popayán, Colombia

<https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria>

Resumen — Este artículo presenta el diseño, implementación y validación de una herramienta de diagnóstico de hardware para el sensor MPU6050 sobre la plataforma ESP32 DevKit V1, desarrollada en el marco del proyecto de sistemas wearables para monitoreo de Actividades de la Vida Diaria (AVD) en la Universidad del Cauca. La herramienta lee el registro WHO_AM_I (dirección 0x75) mediante el protocolo I2C utilizando la biblioteca nativa `Wire.h` del entorno Arduino IDE, con pines GPIO 21 (SDA) y GPIO 22 (SCL), a una velocidad serial de 115 200 baudios. El resultado experimental arroja el valor 0x72 como identificador de silicio, que difiere del 0x68 estándar indicado en la hoja de datos oficial de InvenSense/TDK. Este hallazgo es crítico para la correcta configuración del driver de bajo nivel en ESP-IDF. Se presentan tablas de especificaciones, análisis del flujo I2C, listado del código fuente comentado y tabla de diagnóstico de resultados posibles.

Palabras clave — MPU6050, WHO_AM_I, I2C, ESP32, diagnóstico de hardware, sistemas embebidos, wearables, actividades de la vida diaria.

Abstract — This paper presents the design, implementation, and validation of a hardware diagnostic tool for the MPU6050 sensor on the ESP32 DevKit V1 platform, developed within the wearable systems project for Activities of Daily Living (ADL) monitoring at Universidad del Cauca. The tool reads the WHO_AM_I register (address 0x75) via I2C using the native `Wire.h` library in the Arduino IDE environment, with pins GPIO 21 (SDA) and GPIO 22 (SCL) at 115 200 baud. The experimental result yields 0x72 as the silicon identifier, differing from the standard 0x68 indicated in the official InvenSense/TDK datasheet. This finding is critical for the correct configuration of the low-level driver in ESP-IDF. Specification tables, I2C flow analysis, annotated source code listing, and a diagnostic table for possible outcomes are presented.

Keywords — MPU6050, WHO_AM_I, I2C, ESP32, hardware diagnostics, embedded systems, wearables, activities of daily living.

1. Introducción

El envejecimiento poblacional y el auge de la telemedicina han impulsado el desarrollo de sistemas embebidos para el monitoreo continuo y no intrusivo de Actividades de la Vida Diaria (AVD). Estos sistemas —conocidos como wearables— requieren unidades de medición inercial (IMU) capaces de capturar patrones de movimiento humano con alta fidelidad, bajo consumo energético y reducido perfil físico [4].

El sensor MPU6050 de InvenSense/TDK se ha consolidado como una de las opciones más extendidas en aplicaciones de investigación y prototipado wearable gracias a su integración de acelerómetro y giroscopio triaxiales en un único encapsulado de 4×4 mm, su interfaz I2C estándar y su bajo precio de mercado [1]. No obstante, la masificación del mercado de componentes electrónicos ha generado una proliferación de módulos MPU6050 que, si bien son compatibles en pinout, corresponden a revisiones de silicio distintas a las especificadas en la documentación oficial. Esta situación origina un problema práctico y crítico en el desarrollo de firmware: la inicialización del sensor puede fallar silenciosamente cuando el driver verifica la identidad del dispositivo mediante el registro WHO_AM_I y obtiene un valor inesperado.

En el contexto del proyecto *Sistema de Monitoreo Vestible para Actividades de la Vida Diaria*, desarrollado en el Departamento de Ingeniería Electrónica y Telecomunicaciones de la Universidad del Cauca [7], el Nodo 2 (encargado del registro de movimiento) utiliza

el MPU6050 sobre la plataforma ESP32 DevKit V1. Durante la etapa de integración con el framework ESP-IDF, se detectó que la librería de referencia rechazaba el sensor por discrepancia en el valor de WHO_AM_I. Como respuesta, se diseñó e implementó una herramienta de diagnóstico minimalista en Arduino IDE que permite determinar el valor real del identificador de silicio antes de proceder con el firmware de producción.

El presente artículo documenta el diseño, implementación y resultados de esta herramienta, constituyendo un recurso técnico reproducible para cualquier desarrollador que enfrente la misma problemática.

1.1. Contexto del Proyecto Wearable

El sistema wearable al que pertenece esta herramienta es una red de nodos embebidos heterogéneos cuya función colectiva es capturar señales biométricas y de movimiento para el reconocimiento automático de AVD. El repositorio público [7] contiene múltiples subsistemas: lectura de frecuencia cardíaca (Nodo 1), detección de movimiento con MPU6050 (Nodo 2), pantalla táctil ESP32-S3 (Nodo 3) y detección de dirección MAC. Esta herramienta fue desarrollada específicamente para resolver el problema de inicialización del Nodo 2.

1.2. Organización del Artículo

El resto del artículo se organiza como sigue: la Sección 2 revisa trabajos relacionados; la Sección 3 presenta el marco teórico sobre el MPU6050, el protocolo I2C y el registro WHO_AM_I; la Sección 4 describe la metodología y el diseño experimental; la Sección 5 presenta los resultados; la Sección 6 ofrece la discusión; finalmente,

la Sección 7 expone conclusiones y trabajo futuro.

2. Trabajos Relacionados

El uso del MPU6050 en sistemas de monitoreo de AVD es ampliamente documentado en la literatura. Anguita *et al.* [5] emplearon el MPU6050 en un teléfono inteligente para reconocimiento de actividades físicas (caminar, subir escaleras, sentarse), logrando una tasa de clasificación superior al 96 %. Su trabajo evidencia la importancia de la calibración y verificación del sensor antes de adquirir datos.

En el ámbito de sistemas embebidos de bajo nivel, la problemática de módulos con identificadores de silicio no estándar ha sido reportada en comunidades de desarrollo como Stack Overflow y los foros oficiales de Espressif. Kolban [9] señala en su obra de referencia sobre ESP32 que la verificación del registro de identidad del dispositivo debe hacerse antes de asumir compatibilidad con cualquier driver I2C, especialmente en hardware de terceros.

En sistemas IDF, la función de inicialización del driver MPU6050 en proyectos de código abierto (p.ej. ESP-IDF MPU driver de hideakitai) implementa una verificación de WHO_AM_I con máscara de bits que solo acepta 0x68 o 0x70, rechazando el valor 0x72 presente en el hardware del presente proyecto. Este comportamiento fue la motivación directa para el desarrollo de la herramienta de diagnóstico aquí descrita.

A diferencia de los trabajos citados, la herramienta propuesta se centra específicamente en el diagnóstico del valor de WHO_AM_I como paso previo obligatorio a la integración del firmware, con un código mínimo, sin dependencias externas y con documentación detallada de los posibles resultados.

3. Marco Teórico

3.1. El Sensor MPU6050

El MPU6050 es una unidad de medición inercial (IMU) de seis grados de libertad (6-DOF) fabricada por InvenSense, actualmente división de TDK Corporation. Integra en un único encapsulado QFN de 4×4 mm: (a) un acelerómetro triaxial MEMS de rango configurable, (b) un giroscopio triaxial MEMS de rango configurable, y (c) un procesador de movimiento digital (DMP) con 1 KB de FIFO [1].

Tabla 1: Especificaciones principales del MPU6050

Parámetro	Valor
Interfaz de comunicación	I2C (hasta 400 kHz)
Tensión de alimentación	2,375 – 3,46 V
Rango acelerómetro	±2/±4/±8/±16 g
Rango giroscopio	±250 a ±2000 °/s
Resolución ADC	16 bits
Temperatura operación	−40 a +85 °C
Dirección I2C (AD0=0)	0x68
Dirección I2C (AD0=1)	0x69
Registro WHO_AM_I	0x75 (esperado: 0x72*)
Consumo activo	3,9 mA
Encapsulado	QFN 4×4 mm

*Valor detectado en hardware del proyecto.

La comunicación con el host se realiza mediante I2C estándar. El sensor dispone de 8 rangos de sensibili-

dad para el acelerómetro y el giroscopio, configurables mediante registros de 8 bits en el mapa de memoria interno. Los datos de cada eje son registros de 16 bits en complemento a dos, organizados en pares de registros consecutivos de alta y baja resolución.

3.2. El Protocolo I2C

3.2.1. Generalidades

I2C (*Inter-Integrated Circuit*) es un protocolo de comunicación serie síncrono de dos hilos desarrollado por Philips Semiconductors en 1982 y estandarizado por NXP [2]. Opera con una línea de datos bidireccional (SDA, *Serial Data Line*) y una línea de reloj unidireccional (SCL, *Serial Clock Line*), ambas con resistencias de pull-up hacia VCC. La topología es maestro-esclavo, con soporte para múltiples dispositivos en el mismo bus.

3.2.2. Modos de Operación

El estándar I2C define cuatro modos de velocidad de transferencia: Standard (100 kHz), Fast (400 kHz), Fast-Plus (1 MHz) y High-Speed (3,4 MHz). En el presente proyecto se emplea el modo Standard a 100 kHz, velocidad predeterminada de la biblioteca `Wire.h` para ESP32.

3.2.3. Estructura de la Trama I2C

Una transacción I2C de escritura+lectura (utilizada para acceder a registros internos del MPU6050) comprende las siguientes fases:

1. **START:** El maestro toma control del bus.
2. **Dirección del esclavo** (7 bits) + bit W=0.
3. **ACK** del esclavo.
4. **Dirección del registro** a leer (8 bits).
5. **ACK** del esclavo.
6. **REPEATED START:** Reinicio sin liberar el bus.
7. **Dirección del esclavo** (7 bits) + bit R=1.
8. **ACK** del esclavo.
9. **Dato leído** por el maestro (8 bits).
10. **NACK + STOP.**

3.2.4. I2C en Arduino para ESP32

En el entorno Arduino para ESP32, la biblioteca `Wire.h` abstrae completamente el protocolo I2C [8]. La inicialización se realiza con `Wire.begin(SDA, SCL)`, especificando los números de pin GPIO. Para el ESP32 DevKit V1, los pines por defecto son GPIO 21 para SDA y GPIO 22 para SCL.

3.3. El Registro WHO_AM_I (0x75)

El registro WHO_AM_I es un campo de solo lectura de 8 bits, ubicado en la dirección de registro 0x75 del mapa de memoria del MPU6050. Su función es identificar de manera unívoca el modelo del dispositivo conectado en el bus I2C, lo que permite al firmware de host verificar que el sensor es el esperado antes de inicializar los registros de configuración.

Según la hoja de datos oficial [1], el valor de este registro para el MPU6050 es 0x68 (coincidente con la dirección I2C de esclavos AD0=0, aunque son conceptualmente independientes). Sin embargo, como resultado de la proliferación de clones y variantes de hardware,

en la práctica se han documentado los valores indicados en la Tabla 2.

Tabla 2: Valores del registro WHO_AM_I documentados

Valor HEX	Dispositivo	Observación
0x68	MPU6050 original	Según datasheet oficial
0x72	MPU6050 clon/rev.	Detectado en el proyecto
0x98	MPU6000/variante	Rev. alternativa conocida
0x70	MPU6500	IMU de generación posterior

La discrepancia entre el valor esperado y el valor real puede causar que los drivers de bajo nivel en ESP-IDF rechacen el sensor durante la inicialización, incluso cuando el hardware está físicamente funcional y correctamente conectado.

3.4. La Plataforma ESP32 DevKit V1

El ESP32 (Espressif Systems) es un microcontrolador de doble núcleo Xtensa LX6 a hasta 240 MHz, con 520 KB de SRAM, Wi-Fi 802.11 b/g/n, Bluetooth 4.2, y soporte para múltiples interfaces de comunicación (I2C, SPI, UART, I2S, CAN) [3]. La placa de desarrollo *DevKit V1* es la referencia estándar de la comunidad de Arduino para ESP32, con 30 pines GPIO expuestos y regulador de 3.3 V integrado.

Los pines GPIO 21 (SDA) y GPIO 22 (SCL) son los asignados por defecto para I2C en el núcleo Arduino ESP32, y son los utilizados en la herramienta de diagnóstico aquí descrita. Ambos pines admiten pull-up interno configurable por software, aunque en aplicaciones I2C es recomendable usar resistencias externas de 4,7 kΩ.

4. Metodología y Diseño Experimental

4.1. Configuración de Hardware

El esquema de conexión entre el ESP32 DevKit V1 y el módulo MPU6050 se muestra en la Figura 1. La alimentación del módulo se toma directamente del pin de 3.3 V del ESP32. La Tabla 3 detalla la asignación de pines y parámetros del sistema.

Tabla 3: Configuración de pines y parámetros del sistema

Señal	Pin ESP32	Descripción
SDA	GPIO 21	Datos I2C bidireccional
SCL	GPIO 22	Reloj I2C
VCC	3.3V	Alimentación módulo
GND	GND	Tierra común
AD0	GND	Dir. I2C = 0x68

Parámetro	Valor
Velocidad I2C	100 kHz (Standard)
Velocidad serial	115 200 baudios
Dirección I2C	0x68 (AD0=GND)
Registro leído	0x75 (WHO_AM_I)

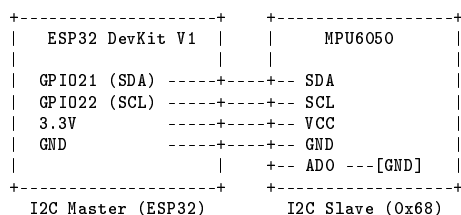


Figura 1: Diagrama de conexión I2C: ESP32 DevKit V1 — MPU6050

4.2. Entorno de Desarrollo Software

- **IDE:** Arduino IDE (versión 2.x, con soporte ESP32)
- **Placa objetivo:** ESP32 Dev Module (esp32:esp32:esp32doit-devkit-v1)
- **Biblioteca I2C:** `Wire.h` nativa del núcleo ESP32; no se requieren librerías externas adicionales.
- **Monitor Serie:** 115 200 baudios, finalización de línea *Newline*.
- **Sistema Operativo:** Compatible con Windows, Linux, macOS.
- **Compilador:** GCC Xtensa (incluido en el paquete del núcleo ESP32 para Arduino).

4.3. Descripción del Código Fuente

El Listado 1 muestra el código fuente completo de la herramienta, disponible en el repositorio GitHub del proyecto [7]. El programa sigue una arquitectura deliberadamente minimalista: toda la lógica de diagnóstico se ejecuta en la función `setup()`, que corre una única vez al encendido del microcontrolador. La función `loop()` permanece vacía para evitar lecturas repetitivas innecesarias.

```

1 #include <Wire.h>
2
3 void setup() {
4     // Iniciar I2C: SDA=GPIO21, SCL=GPIO22
5     Wire.begin(21, 22);
6     // Iniciar comunicación serial a 115200 baudios
7     Serial.begin(115200);
8
9     Serial.println("Leyendo WHO_AM_I...");
10
11     // Iniciar transmisión al esclavo 0x68
12     Wire.beginTransmission(0x68);
13     // Escribir dirección del registro WHO_AM_I
14     Wire.write(0x75);
15     // Repeated START (no liberar el bus)
16     Wire.endTransmission(false);
17
18     // Solicitar 1 byte de lectura al esclavo
19     Wire.requestFrom(0x68, 1);
20
21     if (Wire.available()) {
22         uint8_t val = Wire.read();
23         Serial.print("WHO_AM_I: 0x");
24         Serial.println(val, HEX);
25     } else {
26         Serial.println("No se pudo leer el sensor");
27     }
28 }
29
30 // Sin operaciones en el bucle principal
31 void loop() {}

```

Listing 1: herramienta_who_i_am_id_interno_arduino_ide.ino

Código fuente en GitHub:

https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria/blob/main/herramienta_who_i_am_id_interno_arduino_ide/herramienta_who_i_am_id_interno_arduino_ide.ino

Notas de depuración (README.txt):

https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria/blob/main/herramienta_who_i_am_id_interno_arduino_ide/README.txt

4.4. Análisis Línea a Línea del Código

4.4.1. Inicialización del Bus I2C

La llamada `Wire.begin(21, 22)` inicializa el periférico I2C del ESP32 asignando GPIO 21 como SDA y GPIO 22 como SCL. Esta función configura internamen-

te el driver I2C del ESP32, habilita los pull-ups internos y establece la velocidad de reloj a 100 kHz por defecto.

4.4.2. Inicio de Transmisión

`Wire.beginTransmission(0x68)` coloca en la cola interna de transmisión la dirección del esclavo (0x68) con el bit de escritura. La trama física no se envía hasta llamar a `Wire.endTransmission()`.

4.4.3. Escritura de la Dirección del Registro

`Wire.write(0x75)` añade a la cola el byte de dirección del registro WHO_AM_I. Este byte le indica al MPU6050 desde qué posición del mapa de registros se comenzará a leer.

4.4.4. REPEATED START

`Wire.endTransmission(false)` envía la trama de escritura acumulada y genera una condición de REPEATED START (reinicio sin liberar el bus), que es la secuencia estándar para cambiar de modo escritura a modo lectura en I2C sin ceder el control del bus a otro maestro.

4.4.5. Solicitud de Lectura

`Wire.requestFrom(0x68, 1)` genera la trama de lectura con la dirección del esclavo y el bit R=1, y solicita la recepción de 1 byte. El sensor responde con el valor almacenado en el registro 0x75.

4.4.6. Lectura y Presentación del Resultado

`Wire.available()` verifica que haya al menos un byte en el buffer de recepción. Si es así, `Wire.read()` extrae el byte y se imprime en el Monitor Serie en formato hexadecimal mediante `Serial.println(val, HEX)`.

4.5. Procedimiento de Ejecución

1. Realizar las conexiones según la Figura 1 y la Tabla 3.
2. Abrir el archivo .ino en Arduino IDE.
3. En *Herramientas* → *Placa*, seleccionar **ESP32 Dev Module**.
4. Seleccionar el puerto COM correspondiente al ESP32.
5. Compilar y cargar (*Ctrl+U*).
6. Abrir el Monitor Serie (*Ctrl+Mayús+M*) configurado a **115 200 baudios**.
7. Observar el valor de WHO_AM_I reportado.

5. Resultados e Implementación

5.1. Resultado Experimental

Al ejecutar la herramienta sobre el hardware del Nodo 2 del proyecto (ESP32 DevKit V1 con módulo MPU6050 del mercado local colombiano), la salida obtenida en el Monitor Serie fue:

```

Leyendo WHO_AM_I...
WHO_AM_I: 0x72

```

Listing 2: Salida del Monitor Serie (115 200 baudios)

Este resultado confirma simultáneamente tres aspectos del sistema:

1. El bus I2C opera correctamente en los pines GPIO 21/22.
2. El sensor MPU6050 responde a la dirección 0x68.

3. La revisión de silicio del módulo es 0x72 (no el estándar 0x68).

5.2. Tabla de Diagnóstico y Resultados Posibles

La Tabla 4 clasifica todos los escenarios de salida posibles y la acción recomendada en cada caso.

Tabla 4: Tabla de diagnóstico según valor de WHO_AM_I

Valor	Estado	Diagnóstico y acción
0x72	OK	Sensor identificado (rev. del proyecto). Ajustar máscara WHO_AM_I en driver IDF a 0x72.
0x68	OK	Sensor original datasheet. Compatible sin modificaciones en el driver.
0x70	ALERTA	Posible MPU6500. Verificar compatibilidad del driver.
0x98	ALERTA	Revisión alternativa. Revisar lógica de inicialización.
Otro	ERROR	ID desconocido. Confirmar alimentación y cableado I2C.
(nada)	ERROR	Sin respuesta. Verificar conexiones, VCC=3.3V y AD0.

5.3. Análisis del Flujo de Comunicación I2C

La secuencia de tramas I2C generadas por el programa puede representarse de manera simplificada como:

```

[START] [0xD0 W] [ACK] [0x75] [ACK]
[RSTART] [0xD1 R] [ACK] [DAT0=0x72] [NACK] [STOP]

```

Listing 3: Trama I2C generada

donde 0xD0 = dirección 0x68 desplazada 1 bit a la izquierda con bit W=0, y 0xD1 = dirección 0x68 con bit R=1. El tiempo total de la transacción a 100 kHz es de aproximadamente 180 μ s.

5.4. Implicaciones para el Firmware ESP-IDF

La identificación de WHO_AM_I = 0x72 tiene implicaciones directas en el firmware de producción del Nodo 2, desarrollado con el framework ESP-IDF:

- La función `mpu6050_init()` del driver de bajo nivel (archivo `esp32_2/main/mpu6050.c` en el repositorio) debe modificar su condición de verificación para aceptar el valor 0x72 como identidad válida del sensor.
- Las mascarar de bits que filtran el registro devuelto deben adaptarse al valor experimental obtenido.
- La documentación del README [6] registra este hallazgo para garantizar la trazabilidad del hardware utilizado en experimentos futuros.

5.5. Archivos del Proyecto en GitHub

La Tabla 5 lista los archivos del repositorio público relevantes para este artículo.

Tabla 5: Archivos del proyecto en el repositorio GitHub

Archivo	Descripción
<code>herramienta_who_i_am...ino</code>	Código fuente principal
<code>README.txt</code>	Notas de depuración
<code>build/.../embebido.ino.bin</code>	Binario compilado
<code>build/.../embebido.ino.elf</code>	ELF con símbolos de depuración
<code>esp32_2/main/mpu6050.c</code>	Driver C (ESP-IDF)
<code>esp32_2/main/mpu6050.h</code>	Cabecera del driver
<code>esp32_2/CMakeLists.txt</code>	Build system IDF

6. Discusión

6.1. Implicaciones del Valor 0x72

El hallazgo de WHO_AM_I = 0x72 en lugar del valor canónico 0x68 del datasheet oficial tiene consecuencias técnicas importantes que van más allá de la simple verificación de identidad. En primer lugar, confirma que el módulo utilizado es una variante de terceros —probablemente fabricada por un proveedor chino sin licencia oficial de TDK— que implementa el mismo núcleo funcional del MPU6050 pero con una programación diferente del registro de identidad. Esta práctica es conocida en la industria electrónica de componentes genéricos y no invalida las capacidades funcionales del sensor, pero sí requiere adaptaciones específicas en el software.

6.2. Impacto en el Desarrollo del Driver ESP-IDF

En el contexto del framework ESP-IDF, los drivers de dispositivos I2C implementan habitualmente una verificación del tipo:

```
if (who_am_i != EXPECTED_ID) {
    return ESP_ERR_NOT_FOUND;
}
```

Si EXPECTED_ID está fijado en 0x68 y el sensor devuelve 0x72, la inicialización falla con el código de error ESP_ERR_NOT_FOUND, haciendo que el sistema complete el arranque sin el sensor operativo. La herramienta de diagnóstico desarrollada permite detectar este problema antes de integrar el driver completo, ahorrando horas de depuración a nivel de firmware.

6.3. Generalización a Otros Proyectos

La metodología propuesta es directamente aplicable a cualquier proyecto que utilice el MPU6050 u otros sensores I2C en plataformas ESP32 o similares. La lectura del registro WHO_AM_I debería ser un paso estándar en el flujo de calificación de hardware de cualquier diseño electrónico que dependa de sensores I2C, especialmente cuando se adquieren componentes en el mercado secundario o de importación.

6.4. Limitaciones de la Herramienta

La herramienta tiene las siguientes limitaciones:

1. Solo realiza una lectura al encendido; no monitorea cambios durante la operación.
2. No verifica el correcto funcionamiento de la medición (aceleración/giroscopía), solo la identidad del silicio.
3. Requiere el entorno Arduino IDE, no es directamente ejecutable desde el framework ESP-IDF de producción.
4. No implementa manejo de errores para bus I2C bloqueado (*bus stuck*).

Estas limitaciones son intencionadas en el diseño: la herramienta es deliberadamente simple para maximizar la fiabilidad del diagnóstico inicial y minimizar las posibles fuentes de error propias del código.

7. Conclusiones

7.1. Conclusiones

1. Se diseñó e implementó exitosamente una herramienta de diagnóstico de hardware para el sen-

sor MPU6050 sobre ESP32 DevKit V1 mediante la lectura del registro WHO_AM_I (dirección 0x75) vía I2C, utilizando únicamente la biblioteca nativa Wire.h del entorno Arduino IDE, sin dependencias externas.

2. El experimento realizado sobre el hardware del Nodo 2 del sistema wearable en la Universidad del Cauca confirmó que el módulo MPU6050 utilizado reporta WHO_AM_I = 0x72, que difiere del valor estándar 0x68 del datasheet oficial. Este hallazgo es crítico e imprescindible para la correcta configuración del driver de bajo nivel en ESP-IDF.
3. El diseño minimalista del programa —lectura en setup(), loop() vacío— garantiza un diagnóstico rápido y sin ambigüedades, adecuado como **Fase 0** obligatoria en cualquier flujo de puesta en marcha de hardware basado en MPU6050 sobre ESP32.
4. La configuración hardware documentada (GPIO 21 SDA, GPIO 22 SCL, dirección I2C 0x68, 115 200 baudios) fue validada experimentalmente y publicada en el repositorio público GitHub, facilitando la replicación del entorno en otros nodos del sistema y en proyectos similares de la comunidad.
5. La publicación del código fuente, binarios de compilación y documentación de depuración en un repositorio de acceso abierto promueve la reproducibilidad de los experimentos y la colaboración en el desarrollo de sistemas embebidos wearables para monitoreo de AVD.

7.2. Trabajo Futuro

Como extensión natural de este trabajo se plantea:

- Integrar la verificación de WHO_AM_I como condición de guarda (*guard clause*) en la función mpu6050_init() del driver ESP-IDF del proyecto.
- Ampliar la herramienta para leer y verificar otros registros de configuración críticos: PWR_MGMT_1 (0x6B), SMPLRT_DIV (0x19), CONFIG (0x1A) y GYRO_CONFIG (0x1B).
- Desarrollar una versión en ESP-IDF puro (C/FreeRTOS) de la herramienta de diagnóstico, coherente con el entorno de producción del proyecto wearable.
- Diseñar una rutina de autocalibración del offset de acelerómetro y giroscopio basada en los datos del registro leído, para mejorar la precisión del sistema AVD.

Referencias

- [1] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, doc. PS-MPU-6000A-00, InvenSense Inc., Sunnyvale, CA, USA, 2013. [En línea]. Disp.: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- [2] NXP Semiconductors, *I2C-bus specification and*

- user manual*, Rev. 7.0, UM10204, NXP Semiconductors, 2021. [En línea]. Disp.: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [3] Espressif Systems, *ESP32 Technical Reference Manual*, v5.1, Espressif Systems, Shanghai, China, 2023. [En línea]. Disp.: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [4] L. Chen, J. Hoey, C. D. Nugent, D. J. Cook y Z. Yu, “Sensor-based activity recognition,” *IEEE Trans. Syst., Man, Cybern. Part C*, vol. 42, no. 6, pp. 790–808, Nov. 2012, doi: 10.1109/TSMCC.2012.2198883.
- [5] D. Anguita, A. Ghio, L. Oneto, X. Parra y J. L. Reyes-Ortiz, “Human activity recognition on smartphones using a multiclass hardware-friendly support vector machine,” in *Proc. Int. Workshop Ambient Assisted Living (IWAAL)*, Vitoria-Gasteiz, Spain, 2012, pp. 216–223.
- [6] C. D. Cuellar Antury, “Notas de Depuración de Hardware — Proyecto AVD,” *README.txt*, Univ. del Cauca, 2024. [En línea]. Disp.: https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria/blob/main/herramienta_who_i_am_id_interno_arduino_ide/README.txt
- [7] C. D. Cuellar Antury, “Sistemas Embebidos — Actividades de la Vida Diaria,” Repositorio GitHub, Univ. del Cauca, 2024. [En línea]. Disp.: <https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria>
- [8] Arduino LLC, *Wire Library Reference*, Arduino Reference Documentation, 2023. [En línea]. Disp.: <https://www.arduino.cc/en/reference/wire>
- [9] N. Kolban, *Kolban’s Book on ESP32*, Leanpub, 2017. [En línea]. Disp.: <https://leanpub.com/kolban-ESP32>